

ADAMO LAB

Coding & Data Management Guidelines

The adopted handbook — 33 rules

Ratified by lab vote, 2026-07-08
Eric, Rachel Morris, Lyndon Rakusen, Brandon

Contents

1	How this handbook was made	3
2	Repository & Project Structure	4
3	Version Control	6
4	Environments & Reproducibility	7
5	Coding Conventions	9
6	Naming Conventions	11
7	Data Management	14
8	Collaboration & Handover	16
A	Naming at a glance	17
B	Checking a repository against these rules	18
C	Proposed and vetoed	19

Chapter 1

How this handbook was made

Every rule here was proposed, put to the lab, and adopted by consensus. The bar is unanimity: leaving a question blank counts as agreement, but a single objection drops the rule. A rule one member refuses to follow is not a shared standard, and the lab would rather keep a smaller set that everyone actually follows.

40 rules were proposed, 33 adopted, 7 blocked. The blocked ones are listed in the last appendix with who objected, so nothing is quietly buried.

Ratified 2026-07-08 by Eric, Rachel Morris, Lyndon Rakusen, Brandon.

Chapter 2

Repository & Project Structure

S1 One repo per project, not per person

The work survives the author leaving.

Code belongs to the project, not to whoever wrote it. When someone graduates, their scripts stay findable and runnable instead of disappearing onto a personal laptop.

```
adamolab/                # one GitHub org
├─ reaction-time-study/   # one repo per project
├─ eye-tracking-2026/
└─ saccade-model/
```

S2 Standard top-level layout

You find things without asking anyone.

Give every repo the same skeleton: README, data/, src/, results/, and an environment lock. Open any project and you know where the code, data pointers, and outputs live.

```
saccade-model/
├─ README.md
├─ data/           # or a pointer to storage
├─ src/
├─ results/
└─ environment.yml
```

S3 No data committed to git

Git chokes on large binaries.

Git diffs text well and binaries badly. Commit a few gigabytes and clones crawl, history bloats, and you cannot rewind. Keep data in real storage and version it with a tool built for the job.

```
# .gitignore
data/
*.h5

# version data with DVC instead of git
dvc add data/raw.h5
```

S4 README answers three questions

New members onboard without you.

Cover what the project is, how to set up the environment, and how to run it end to end. A newcomer should reach a first result without messaging you.

Saccade Model

****What:**** fits saccade latency from eye-tracking data.

****Setup:**** `conda env create -f environment.yml`

****Run:**** `make all`

Chapter 3

Version Control

V2 was proposed for this chapter and vetoed. See the appendix.

V1 All project code under git

The throwaway script made a figure.

The one-off that produced Figure 3 is the one you need again for the revision. Track everything from the first commit. It costs nothing and saves the panic later.

```
git init
git add analysis.py figures.R
git commit -m "Initial analysis pipeline"
```

V3 Main branch always runs

A colleague may pull at any moment.

Keep main runnable at all times. Do risky work on a branch and merge only once it passes. Whoever pulls next inherits working code, not your half-finished state.

```
git switch -c fix-outlier-filter
# ...work, run tests...
git switch main
git merge fix-outlier-filter # only once green
```

V4 Shared remote org, not personal accounts

Access outlives any one person.

Host repos under a lab organization instead of a personal account. Permissions and continuity then survive graduations, and nobody chases a former student for admin rights.

```
# lab org remote, not a personal account
git remote add origin \
  git@github.com:adamolab/saccade-model.git
```

Chapter 4

Environments & Reproducibility

E1 Every project pins its environment

'Works on my machine' is not a result.

Record exact package versions so the analysis runs the same next year and on a colleague's laptop. Leave the environment unpinned and it drifts until the numbers stop matching.

```
# environment.yml
name: saccade
dependencies:
  - python=3.11
  - numpy=1.26.4
  - pandas=2.2.1
```

E2 One command reproduces the main result

Untested reproducibility is a guess.

Wire the pipeline behind a single entry point: a Makefile, a script, or a workflow. If rerunning the analysis needs tribal knowledge, it does not reproduce.

```
all: results/fig1.pdf

results/fig1.pdf: src/plot.py data/clean.csv
    python src/plot.py
```

E3 Set and record a random seed

Same input should give the same figure.

Fix and log the seed wherever randomness touches results: splits, initialization, sampling. Skip it and two identical runs disagree, and you cannot tell signal from noise.

```
import numpy as np, torch
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
```

E4 Scripts or clean notebooks for published results

Out-of-order cells lie.

Explore in notebooks, but generate any published number from a script or a notebook that runs top to bottom without error. Hidden execution order is where mistakes hide.

```
# run a notebook top-to-bottom, no manual state
jupyter nbconvert --to notebook --execute \
  --output out.ipynb analysis.ipynb
```


Chapter 5

Coding Conventions

C1 Auto-formatter per language

No human should format code.

Hand formatting to a tool so nobody burns a review comment on whitespace. Consistent style also keeps diffs focused on real changes instead of reflows.

```
black src/                                # Python
Rscript -e 'styler::style_dir("R")'      # R
shfmt -w scripts/                        # Shell
```

C2 Descriptive names beat comments that explain bad names

A good name never goes stale.

subject_reaction_times explains itself. srt needs a comment, and that comment drifts from the code the moment someone edits nearby.

```
# bad
srt = df[df.srt > 0.1]
# good
valid = trials[trials.reaction_time > 0.1]
```

C3 No hard-coded absolute paths

/Users/alice/ breaks for everyone else.

Absolute paths fail the instant a second person runs the code. Use relative paths or one path variable at the top so the project moves between machines.

```
# bad
df = pd.read_csv("/Users/alice/data/rt.csv")
# good
from pathlib import Path
DATA = Path(__file__).parent / "data"
df = pd.read_csv(DATA / "rt.csv")
```

C4 Functions for anything used more than twice

Fix it once, not in five places.

Copy, paste, tweak, and now four near-copies drift apart. Pull anything you write a third time into one function so a fix lands everywhere at once.

```
def zscore(x):  
    return (x - x.mean()) / x.std()  
  
rt_z = zscore(df.rt)  
amp_z = zscore(df.amp)
```

C5 Config at the top, not scattered

Changing a run should take one edit.

Keep parameters, paths, and thresholds in one block or file instead of sprinkled through the code. Tuning an experiment then means editing one spot.

```
# config.py  
RAW = "data/raw.csv"  
RT_MIN = 0.1  
N_BOOT = 10_000
```

Chapter 6

Naming Conventions

N3, N13 were proposed for this chapter and vetoed. See the appendix.

N1 Python identifiers follow PEP 8

The linter and every reader expect it.

snake_case for variables, functions, and modules. PascalCase for classes. UPPER_SNAKE_CASE for constants.

```
rt_min = 0.1           # variable: snake_case
RT_MAX = 2.0           # constant: UPPER_SNAKE
def clean_trials(raw): ... # function: snake_case
class SaccadeModel: ... # class: PascalCase
```

N2 R identifiers follow the tidyverse style

A dot means an S3 method, so avoid dots.

snake_case for variables and functions. Skip dots in new names. PascalCase for R6 and S4 classes.

```
rt_min <- 0.1           # snake_case
clean_trials <- function(raw) { } # snake_case
# avoid: clean.trials (dot = S3 method)
```

N4 Shell: snake_case locals, UPPER_SNAKE exported

Hyphens are illegal in shell variable names.

Local variables and functions use snake_case. Environment and exported variables use UPPER_SNAKE_CASE.

```
DATA_ROOT=/mnt/labshare # exported: UPPER_SNAKE
clean_trials() {          # function: snake_case
  local raw_file="$1"     # local: snake_case
}
```

N5 Constants and classes are cross-language

Keep the shouted categories recognizable.

Constants and environment variables are UPPER_SNAKE_CASE in every language. Classes and types are PascalCase in every language.

UPPER_SNAKE_CASE	constants, env vars	(all languages)
PascalCase	classes, types	(all languages)

N6 **Filesystem names are lowercase**

Data/ and data/ collide across machines.

Lowercase every file and directory name. Linux sees two folders where macOS sees one, and that gap corrupts a shared repo.

```
mkdir raw-data      # good
mkdir Raw_Data      # collides on case-insensitive macOS
```

N7 **No spaces in names**

Spaces force quoting and break scripts.

Use a hyphen or underscore instead of a space in any file or directory name.

```
eye-tracking-data.csv    # good
eye tracking data.csv    # needs quoting everywhere
```

N8 **Restrict names to a-z 0-9 - _ .**

Other characters are illegal or shell-special.

Colons and backslashes are illegal on Windows. The characters & * ? ! carry shell meaning. A leading hyphen looks like a command flag.

```
s01_congruent_v02.csv    # good
s01:congruent (2)!.csv   # illegal on Windows, shell-hostile
```

N9 **Avoid reserved names, keep paths short**

Windows chokes on con, nul, and long paths.

Never name a file con, prn, aux, nul, com1, or lpt1. Keep total path length well under 260 characters.

```
reserved on Windows: con prn aux nul com1..com9 lpt1..lpt9
keep the full path under 260 characters
```

N10 **Repositories and branches use kebab-case**

Matches GitHub, URLs, and the ecosystem.

Name repositories and git branches in lowercase kebab-case.

```
adamolab/eye-tracking-2026    # repo
git switch -c fix-outlier-filter # branch
```

N11 Directories use kebab-case

Readable in a URL and a terminal.

Name directories in lowercase kebab-case.

```
saccade-model/
├─ raw-data/
├─ figure-scripts/
└─ clean-output/
```

N12 Script files follow language import rules

Python cannot import a hyphenated module.

Python and R modules use snake_case so they stay importable. Standalone shell tools use kebab-case, matching how you type the command.

```
data_cleaning.py    # Python module, importable
clean_data.R        # R script
run-analysis.sh     # shell tool, a CLI command
```

N14 Config keys use snake_case

They map straight onto code variables.

Keys in YAML, TOML, or JSON use snake_case so they line up with the Python and R variables that read them.

```
# config.yml
rt_min: 0.1
n_boot: 10000
data_root: /mnt/labshare
```

Chapter 7

Data Management

D1, D4, D6 were proposed for this chapter and vetoed. See the appendix.

D2 Consistent, sortable file naming

final_final2 is where projects die.

Name files so machines can read them: ISO dates, no spaces, zero-padded versions. They then sort correctly and a script can glob them.

```
2026-07-08_s01_congruent_v02.csv
2026-07-08_s02_incongruent_v02.csv
# not:  final data (2)!!.csv
```

D3 Every dataset has a data dictionary

Data without metadata turns to noise.

Ship a description of each field with the data: meaning, units, collection date, and who collected it. Skip it and your own spreadsheet baffles you within a year.

```
# data_dictionary.md
rt          Reaction time, seconds
accuracy    1 = correct, 0 = error
subject     Anonymized ID (s01..s40)
```

D5 3-2-1 backup for irreplaceable raw data

Irreplaceable means irreplaceable.

Keep three copies, on two kinds of media, with one off-site. Data you cannot recollect earns at least a primary plus one automated backup.

```
copy 1  /mnt/labshare/saccade      primary
copy 2  external HDD              weekly rsync
copy 3  institutional cloud        offsite
```

D7 Sensitive data documented and access-controlled

Law and ethics require it.

De-identify personal data where you can, lock down access where you cannot, and write down how you handle it.

```
python deidentify.py raw/ -o safe/  
chmod 700 safe/          # owner-only access
```

Chapter 8

Collaboration & Handover

H2 was proposed for this chapter and vetoed. See the appendix.

H1 Leaving-the-lab checklist

Most lost work traces to a departure.

Before anyone leaves: push the code, pin the environment, document where the data lives, and update the README. The alternative is a low bus factor and a scramble after they are gone.

```
[ ] code pushed to origin  
[ ] environment.yml committed  
[ ] data location in README  
[ ] README run steps current
```


Appendix A

Naming at a glance

Level	Case	Example
Repository	kebab-case	eye-tracking-2026
Git branch	kebab-case	fix-outlier-filter
Directory	kebab-case	raw-data/
Python / R script	snake_case	data_cleaning.py
Shell tool	kebab-case	run-analysis.sh
Variable, function (Python, R, shell-local)	snake_case	reaction_time
Class, type	PascalCase	SaccadeModel
Constant, env var	UPPER_SNAKE_CASE	RT_MIN
Config key	snake_case	rt_min

Levels covered by a vetoed rule are absent from this table: the lab set no standard for them.

Appendix B

Checking a repository against these rules

The handbook is published as a plain-text file an agent can read whole: <https://adamo-lab.github.io/guidelines/llms.txt>. Point a coding agent at it and ask for an audit rather than a fix, so you keep control of the changes.

B.1 From a terminal

```
claude "Read https://adamo-lab.github.io/guidelines/llms.txt, then
audit this repository against every rule. For each rule ID give
PASS, FAIL, or N/A with the file or command that proves it.
Do not change any files."
```

Or keep a local copy in the project so every session starts with the rules:

```
curl -o LAB-GUIDELINES.md https://adamo-lab.github.io/guidelines/llms.txt
```

B.2 In a chat window

Paste the contents of `llms.txt`, then your repository tree and README, and ask the same question. Without file access the model can only judge what you show it, so paste the environment lock and the entry-point script too.

B.3 What to expect

Ask for a verdict per rule ID, and for the evidence next to it. The file ends with a **Not adopted** list: those rules were vetoed and must not be reported as failures.

Roughly two thirds of the rules are checkable from the repository alone — layout, git history, pinned environments, naming, hard-coded paths. The rest are facts about the world that no agent can see: whether the backup in D5 ran, who holds access under D7, whether the storage location in the README is still the live one. Those stay a human check.

A clean audit is not a claim that the science is right. It means the project is laid out so the next person can run it.

Appendix C

Proposed and vetoed

These 7 rules did not enter the handbook. Each was blocked by at least one member, and under the consensus rule one objection is enough. They are recorded here with the objection so a later round can pick them up rather than rediscover them.

V2 Commit messages say why, not just what*git log is your real lab notebook.***Blocked by** Brandon.**N3 MATLAB identifiers follow MathWorks style***The file name must match the function.***Blocked by** Lyndon Rakusen.**N13 Data files use the fielded convention***Sort by date, glob by field.***Blocked by** Brandon.**D1 Raw data is read-only, never edited in place***You cannot un-overwrite raw data.***Blocked by** Brandon.**D4 Single source of truth for storage***No more 'which copy is current?'***Blocked by** Brandon.**D6 Provenance link from result to data and code***A reviewer will ask two years later.***Blocked by** Brandon.**H2 Code review for shared or published pipelines***Cheap insurance where it counts.***Blocked by** Lyndon Rakusen.